

RDP+ (RDP Plus) v1.7.0

Author: Glenn Madine | **Released:** 09/15/2026 | **Language:** C++17, Win32 API | **Compiler:** Microsoft C++ 19.51

1. Overview

RDP+ is a small, single-window Windows desktop launcher. It reads a list of hosts (or shell items) from `connections.json` and shows them in a sortable table. Double-clicking a row (or pressing Enter) launches that entry using a command looked up by its **Type** in `actionDefinitions.json`, so the same tool can open RDP sessions, SSH sessions, web pages, Explorer locations, or anything else that can be started from a command line or URL.

Both JSON files live next to the executable and can be edited from inside the app. Changes are picked up automatically the next time the RDP+ window regains focus, with no restart needed.

2. Features

- Data-driven launcher: hosts come from `connections.json`; how each type is launched comes from `actionDefinitions.json`.
- Three-column report view (Hostname or Shell Item, Type, Description) with full-row select and grid lines.
- Click any column header to sort; click again to reverse. Sort arrows are shown in the header.
- Launch by double-click or Enter.
- URL-style actions (`https://`, `http://`, `ftp://`, `file://`, `mailto:`, `tel:`) open through the default handler; everything else runs as an executable with arguments.
- **Launch RDP connection for host not in this list** opens a blank Remote Desktop Connection (`mstsc.exe`).
- **Edit connections.json / Edit actionDefinitions.json** open the file in Notepad, creating it with `[]` if it does not exist.
- **Run...** opens the standard Windows Run dialog (same as Win+R).
- Automatic reload of both JSON files whenever the window is re-activated.
- Full Unicode support: UTF-8 on disk, UTF-16 in the UI.

3. Requirements

- Windows 10 or 11 (any Windows version with ComCtl32 and Shell.Application should work).
- Visual Studio / MSVC toolset with the Windows SDK (built with MSVC 19.51, C++17).
- nlohmann/json single header, saved as `json.hpp` beside the source file.
- `resource.h` defining `IDI_ICON1`, and a resource script compiled to `RDP_Plus.res` (application icon).

4. Building

From a *Developer Command Prompt for VS*, compile the resources, then compile and link:

```
rc RDP_Plus.rc
CL /EHsc /W3 /O2 /GL /DUNICODE /D_UNICODE /DNDEBUG /std:c++17 RDP_Plus.cpp RDP_Plus.res /Fe:RDP_Plus.exe
```

```
/link /SUBSYSTEM:WINDOWS comctl32.lib shell32.lib shlwapi.lib user32.lib gdi32.lib ole32.lib oleaut32.lib
```

Switch	Purpose
/EHsc	Standard C++ exception handling
/W3	Warning level 3
/O2 /GL	Optimize for speed; whole-program optimization
/DUNICODE /D_UNICODE	Build against the wide-character (W) Win32 APIs
/DNDEBUG	Release build (disables assert)
/std:c++17	Needed for std::optional and structured bindings
/SUBSYSTEM:WINDOWS	GUI application (wWinMain entry point, no console)

The `#pragma comment(lib, ...)` lines in the source already pull in comctl32, shell32, shlwapi, ole32 and oleaut32, so the explicit library list on the command line is belt-and-braces.

5. Deployment

Place these files together in one folder:

```
RDP_Plus.exe
connections.json
actionDefinitions.json
```

Because the app creates and edits its JSON files beside the .exe, choose a folder the user can write to (for example a folder under the user profile or a tools share). Under `C:\Program Files` a standard user will not be able to save edits.

6. Configuration files

6.1 connections.json

Each entry is an object with three string fields. Missing fields are left blank.

Field	Meaning
host	Hostname, IP, URL remainder, or shell item (e.g. shell:Downloads). Appended to the action's args or URL prefix.
type	Name of the action to use. Case-insensitive here, because it is upper-cased before lookup.
description	Free text shown in the Description column.

The file can be a flat array:

```
[
  { "host": "dc01.ynext.corp", "type": "rdp", "description": "Domain controller" },
  { "host": "piweb.ynext.corp", "type": "ssh", "description": "Raspberry Pi web server" },
  { "host": "intranet.ynext.corp", "type": "web", "description": "Intranet site" },
  { "host": "shell:Downloads", "type": "explorer", "description": "My Downloads folder" }
]
```

or an object of named groups. Group names are ignored and all entries are flattened into one list (a group value may be a single object or an array):

```
{
```

```

"Servers": [
  { "host": "dc01.ynext.corp", "type": "RDP", "description": "Domain controller" },
],
"Pis": [
  { "host": "piweb.ynext.corp", "type": "SSH", "description": "Raspberry Pi web server" }
]
}

```

6.2 actionDefinitions.json

Must be a top-level JSON array. Every entry must have all three fields, or the file fails to load.

Field	Meaning
action	Action name. Must be UPPERCASE, because the connection's type is upper-cased and then compared exactly.
args	Text placed immediately before the host on the command line. Include any leading space or switch, e.g. " /v:".
command	Executable to run, or a URL prefix such as https://.

```

[
  { "action": "RDP",      "args": " /v:", "command": "C:\\Windows\\System32\\mstsc.exe" },
  { "action": "SSH",      "args": "",    "command": "C:\\Windows\\System32\\OpenSSH\\ssh.exe" },
  { "action": "WEB",      "args": "",    "command": "https://" },
  { "action": "EXPLORER", "args": "",    "command": "explorer.exe" }
]

```

6.3 How a launch is built

- 1 Take the selected row's **type** and upper-case it (**rdp** becomes **RDP**).
- 2 Find the first action whose **action** equals that value, and read its **command** and **args**.
- 3 If **command** starts with **https://**, **http://**, **ftp://**, **file://**, **mailto:** or **tel:**, open **command** + **host** with the default handler (e.g. **https://intranet.ynext.corp**).
- 4 Otherwise run **command** with the argument string **args** + **host** (e.g. **mstsc.exe /v:dc01.ynext.corp**).

7. Using RDP+

```

+-----+
| RDP+ v1.7.0 - To launch, doubleclick a host below |
+-----+-----+-----+
| Hostname or Shell Item | Type | Description |
+-----+-----+-----+
| dc01.ynext.corp       | rdp  | Domain controller |
| piweb.ynext.corp      | ssh  | Raspberry Pi web server |
| ...                   |      |                  |
+-----+-----+-----+
| [Launch RDP connection for host...] [Edit actionDefinitions.json] |
| [Edit connections.json] [ Run... ] [ Exit ] |
+-----+

```

Control	What it does
List (double-click / Enter)	Launches the selected entry using its action.
Column header click	Sorts by that column (case-insensitive); click again to reverse.

Control	What it does
Launch RDP connection for host not in this list	Opens mstsc.exe with no arguments so you can type any host.
Edit actionDefinitions.json	Opens the actions file in Notepad (creates it with [] if missing).
Edit connections.json	Opens the connections file in Notepad (creates it with [] if missing).
Run...	Opens the Windows Run dialog.
Exit	Closes RDP+.

After saving a file in Notepad, just click back on the RDP+ window: both files are reloaded and the list refreshes.

8. Code structure

Area	Key functions / types	Notes
Encoding helpers	Utf8ToWide, WideToUtf8, ToWide, WStringToString, ToUpper	UTF-8 (disk/JSON) to UTF-16 (Win32) conversion.
Action model	struct Action, from_json, to_json	nlohmann/json ADL hooks map JSON objects to Action.
Action loading	ReadFileUtf8, loadActions	Reads bytes with CreateFileW/ReadFile; throws on error or non-array.
Action lookup	findCommandByAction, findArgsByAction	Linear search returning std::optional.
Device model	struct Device, LoadConnections	Tolerant parser: flat array or grouped object; shows a message box on errors.
Config paths	GetExeDir, GetConnectionsJsonPath, GetActionsJsonPath	Both files sit beside the .exe.
Config editing	FileExists, CreateFileWithDefaultContent, OpenJsonFileForEditing	Seeds a missing file with [] then opens Notepad.
Launching	LaunchSomething, OnItemActivated	Chooses URL-open vs. executable + args via ShellExecuteW.
Run dialog	ShowRunDialog	COM Shell.Application IShellDispatch::FileRun (documented API).
List view	PopulateListView, SortDevices, UpdateHeaderSortArrow, ReloadAllData	Fill, sort and refresh the table.
Window	WndProc, wWinMain	Creates controls, lays them out on WM_SIZE, handles commands/notifications, reloads on WM_ACTIVATE.
Unused	Logger, getExeDirectory, UsernameDialog, GetUsernamePopup, WM_LAUNCH	Present but never called (see section 9).

Message flow

- **WM_CREATE:** builds the list view and its three columns, loads both JSON files, creates the five buttons.

- **WM_SIZE**: list view fills the window above two centred button rows (250 px wide buttons; Run and Exit share the right-hand slot of the bottom row).
- **WM_COMMAND**: dispatches the five buttons.
- **WM_NOTIFY**: NM_DBLCLK / NM_RETURN launch the selection; LVN_COLUMNCLICK sorts.
- **WM_ACTIVATE**: on activation, reloads both JSON files and repopulates the list.
- **WM_DESTROY**: posts the quit message.

9. Review notes and suggested improvements

The program is compact and readable. The items below are things noticed while documenting it, roughly in order of impact.

Behaviour

- 1 **Repeating error dialog on bad JSON.** `ReloadAllData` runs on every `WM_ACTIVATE` and shows a message box if either file fails to parse. Dismissing the box re-activates the main window, which reloads and shows the box again, until the file is fixed. Consider reloading only when the file's last-write time has changed, or showing the error once per change.
- 2 **Unknown types fail silently.** If no action matches, the code falls back to the string `"Default"` and calls `ShellExecuteW("Default", ...)`, which fails with no message. `LaunchSomething` also ignores `ShellExecuteW`'s return value. Showing "No action defined for type X" would help.
- 3 **Sort order is lost on reload.** Reloading replaces `g_devices` in file order, but `g_sortCol/g_sortAsc` and the header arrow are kept, so the arrow can show a sort that is no longer applied. Re-apply the current sort after loading (without toggling direction).
- 4 **Selection is lost on every focus change,** because the list is rebuilt each time the window is activated.
- 5 **Hosts with spaces** are appended unquoted to the argument string.

Build and platform

- 1 **Visual styles / sort arrows.** `InitCommonControlsEx` alone does not enable ComCtl32 v6; that needs an application manifest. `HDF_SORTUP/HDF_SORTDOWN` arrows only draw with v6, so make sure `RDP_Plus.res` embeds a manifest (or add `#pragma comment(linker, "/manifestdependency:...")`).
- 2 `#define UNICODE / _UNICODE` in the source duplicate the `/D` switches and can trigger warning C4005 (macro redefinition). Wrap them in `#ifndef`.
- 3 `LoadConnections` converts the path to UTF-8 and opens it with a narrow `std::ifstream`. MSVC interprets narrow paths in the ANSI code page, so a folder with non-ASCII characters would fail. Reusing `ReadFileUtf8` (already used for actions) fixes this and unifies the two loaders.

Tidy-up

- 1 Unused code: `Logger`, `getExeDirectory`, `WStringToString`, `UsernameDialog`, `GetUsernamePopup` (an "SSH Login" prompt) and its leftover `dlgBase/DlgLayout` structs, and the `WM_LAUNCH` constant. Either wire them up or remove them. If `GetUsernamePopup` is kept, its `BYTE dlgMem[512]` buffer should be DWORD-aligned (e.g. `alignas(4)`).

- 2 Duplicate helpers: `ToWide/Utf8ToWide`, `WStringToString/WideToUtf8`, `GetExeDir/getExeDirectory`.
- 3 `PopulateListView` takes an `actions` parameter it never uses.
- 4 `findCommandByAction` and `findArgsByAction` do the same search twice; a single `findAction` returning the whole `Action` would be simpler.
- 5 The `WM_SIZE` comment "Bottom row: Launch RDP / Exit" is out of date (that row is now the top row with Edit `actionDefinitions.json`).
- 6 Both JSON files are loaded twice at start-up (in `WM_CREATE` and on the first `WM_ACTIVATE`).

Security note

`actionDefinitions.json` can run any program with any arguments, so treat it as trusted configuration. Keep the RDP+ folder writable only by the people who should be able to change what the launcher runs.

10. Version

v1.7.0, released 09/15/2026 by Glenn Madine.